DAA Practical 6

0/1 Knapsack Using Dynamic Programming

Top 60 Viva Questions & Answers

Quick formula and concept sheet

State: DP[i][w] = maximum profit using the first i items with capacity w.
Base cases: DP[0][w] = 0 and DP[i][0] = 0.
Recurrence: If wt■ > w, DP[i][w] = DP[i–1][w]. Otherwise DP[i][w] = max(DP[i–1][w], p■ + DP[i–1][w–wt■]).
Time: O(nW). **DP table space:** O(nW).
Sample: weights [10,20,30], profits [60,100,120], capacity 50 → maximum profit 220; selected items 2 and 3.
Method: Bottom-up tabulation. Compare include vs exclude; choose the greater profit.

YouTube links – watch in this order

1. Abdul Bari – 0/1 Knapsack: Two Methods
Open YouTube search – strong conceptual explanation of the DP method.

2. 0/1 Knapsack – Tabulation Method (Hindi)
Open YouTube search – useful for understanding the matrix and dry run.

3. Aditya Verma – 0/1 Knapsack
Watch video – recursion and DP explanation.

4. Dynamic Programming – Principle of Optimality
Watch video – understand why DP works.

A. Knapsack and Dynamic Programming fundamentals

Q1. What is the 0/1 Knapsack problem?

Given items with weights and profits and a knapsack capacity, select a subset of items that maximizes total profit without exceeding the capacity.

Q2. Why is it called 0/1 Knapsack?

Each item has only two choices: 1 means select the whole item and 0 means do not select it. Fractions of an item are not allowed.

Q3. What are the inputs in this practical?

The number of items n, each item's weight, each item's profit, and the knapsack capacity W.

Q4. What is the objective function?

Maximize the sum of selected item profits, subject to the sum of selected weights being at most W.

Q5. What is the difference between 0/1 and Fractional Knapsack?

0/1 Knapsack requires each item to be taken whole or left out. Fractional Knapsack permits taking part of an item.

Q6. What is Dynamic Programming?

Dynamic Programming solves a problem by combining solutions to smaller subproblems and storing their results to avoid repeated computation.

Q7. What two properties make DP suitable for a problem?

Optimal substructure and overlapping subproblems. An optimal larger solution can be built from smaller optimal solutions, and subproblems recur.

Q8. What is optimal substructure in Knapsack?

For a chosen item decision, the remaining items and remaining capacity form a smaller subproblem whose optimal result contributes to the overall optimum.

Q9. What are overlapping subproblems?

The same combinations of remaining items and capacity can be reached through different decision paths, so solving them repeatedly is wasteful.

Q10. Which DP approach does this code use?

Tabulation, a bottom-up approach. It fills a table iteratively, starting with base cases and building toward the full problem.

Q11. What is memoization?

Memoization is top-down DP: solve recursively and store answers to subproblems so they are reused instead of recalculated.

Q12. How is tabulation different from memoization?

Tabulation fills a table iteratively from smaller states upward. Memoization uses recursion and caches states when they are first needed.

B. DP table, recurrence and reconstruction

Q13. What does dp[i][w] represent?

The maximum profit achievable using only the first i items with a knapsack capacity of w.

Q14. Why does the DP table have n+1 rows?

Row zero represents using no items. Rows 1 through n represent considering the first 1 through n items.

Q15. Why does the DP table have W+1 columns?

Column zero represents zero capacity. Columns 1 through W represent every integer capacity up to the knapsack limit.

Q16. Why are all DP cells initially zero?

With zero items or zero capacity, no positive-weight item can be selected, so the maximum profit is zero.

Q17. Why does the outer loop start at i=1?

Row zero is the base case for no items and is already initialized. The algorithm then processes items one by one.

Q18. Why does the inner loop start at w=0?

It calculates the result for every capacity, including zero, and preserves the base-case values in column zero.

Q19. Why does the code use weight[i-1] and profit[i-1]?

The DP rows are numbered from 1 to n, while C++ vectors are indexed from 0 to n–1. Subtracting one maps a DP item row to its vector index.

Q20. What does the exclude statement do?

dp[i][w] = dp[i–1][w] carries forward the best profit from the previous row, representing the choice not to take the current item.

Q21. When can the current item be included?

Only when its weight is less than or equal to the current capacity w.

Q22. Why does the include expression add profit[i-1]?

It adds the profit earned by selecting the current item.

Q23. Why does the include expression use dp[i-1][w-weight[i-1]]?

After selecting the item, it looks up the best profit for the remaining capacity using only earlier items. The previous row prevents reusing the current item.

Q24. Why does the code use max()?

It compares the profit from excluding the item with the profit from including it, then keeps the larger value because the goal is to maximize profit.

Q25. What is the recurrence relation?

If wt■ > w: DP[i][w] = DP[i–1][w]. Otherwise: DP[i][w] = max(DP[i–1][w], p■ + DP[i–1][w–wt■]).

Q26. What are the base cases?

DP[0][w] = 0 for every capacity w, and DP[i][0] = 0 for every number of items i.

Q27. Why does this recurrence guarantee that an item is not selected twice?

The include case refers to row i–1 rather than row i, so the current item cannot be used again within the same decision.

Q28. What is the result for the sample input?

For weights 10, 20, 30; profits 60, 100, 120; and capacity 50, the maximum profit is 220, obtained by selecting items 2 and 3.

Q29. Why are items 2 and 3 selected in the sample?

Their combined weight is 20+30=50, which fits exactly, and their combined profit is 100+120=220, which is greater than the other feasible combinations.

Q30. What does dp[n][capacity] contain?

It contains the maximum profit achievable using all n items with the full knapsack capacity.

Q31. Why does the program trace the table backwards?

The optimal value is in the bottom-right cell. Comparing it with the cell above reveals whether the current item changed the optimal value and was selected.

Q32. What does dp[i][w] != dp[i-1][w] indicate during reconstruction?

For this implementation, it indicates that including item i improved the stored optimum at that state, so the item is selected in the reconstructed solution.

Q33. Why is w reduced after selecting an item?

The selected item's weight is no longer available, so the remaining capacity becomes w–weight[i–1].

Q34. Why are selected items printed in reverse order?

The reconstruction loop starts from item n and moves down to item 1, so it discovers selected items from last to first.

Q35. Can two different optimal selections exist?

Yes. Different subsets can have the same maximum profit. This reconstruction returns one optimal selection according to the table and tie-handling rule.

Q36. What happens if the current item is too heavy for capacity w?

The include option is not evaluated. The DP cell simply keeps the exclude value from the previous row.

C. Complexity and alternative algorithms

Q37. What is the time complexity of this implementation?

O(nW), where n is the number of items and W is the integer capacity, because the table has (n+1)(W+1) states and each is filled in constant time.

Q38. What is the space complexity of the DP table?

O(nW), because it stores (n+1)(W+1) integer values. The input vectors use O(n) additional space.

Q39. Why is O(nW) called pseudo-polynomial?

It is polynomial in the numeric value W, but W may require only log W bits to represent. Therefore the runtime is not necessarily polynomial in the input's bit-length.

Q40. What is brute force for 0/1 Knapsack?

Try every possible subset of n items, calculate its total weight and profit, and keep the best feasible subset.

Q41. What is the time complexity of brute force?

O(2■) subsets are possible; evaluating each subset can take up to O(n), giving a straightforward O(n·2■) implementation.

Q42. Why is DP generally preferable to brute force?

DP reuses solutions to repeated item-and-capacity subproblems instead of evaluating every subset independently.

Q43. Why not use a greedy profit-first strategy?

Choosing the highest individual profit first can block a better combination of smaller items. A locally attractive choice is not always globally optimal.

Q44. Is the greedy profit-to-weight ratio method always optimal for 0/1 Knapsack?

No. It is optimal for the fractional version under the usual assumptions, but it is not guaranteed to solve 0/1 Knapsack optimally.

Q45. What is recursion's role in a recursive Knapsack solution?

It branches into two choices for each item: include it if it fits, or exclude it, then solves the remaining smaller problem.

Q46. Why can plain recursive Knapsack be slow?
It recalculates the same subproblems many times, leading to exponential time in the worst case.

Q47. How does memoization improve recursive Knapsack?
It stores the result for each (item index, remaining capacity) state, avoiding repeated calculations and achieving O(nW) time.

Q48. How can the space usage be reduced to O(W)?
Use a one-dimensional DP array and update capacities in descending order for each item, so the same item is not reused.

Q49. Why must a 1D 0/1 Knapsack DP loop over capacity in descending order?
Descending updates ensure the current item uses values from the previous item iteration. Ascending updates could reuse the same item multiple times, turning it into an unbounded-style update.

Q50. What is Unbounded Knapsack?
A variant in which each item type can be selected any number of times. Its DP update can reuse the current item, unlike 0/1 Knapsack.

Q51. What is one advantage of the 2D table used in this code?
It is easy to understand and makes it straightforward to reconstruct which items were selected by comparing adjacent rows.

Q52. What is one limitation of this implementation?
Its memory use grows with nW, and it assumes non-negative integer weights and capacity. Very large profits may also overflow int.

D. Applications, limitations and code understanding

Q53. How could the code handle large profit values more safely?
Use long long for profit values and DP entries, while still checking that input values and additions remain within the type's range.

Q54. What input validation would improve the program?
Check that n and capacity are non-negative and that all weights and profits satisfy the intended constraints, especially that weights are positive or non-negative as required.

Q55. What are real-world applications of Knapsack-style optimization?
Cargo loading, selecting projects under a budget, allocating limited resources, choosing jobs or tasks under capacity constraints, and selecting indivisible investment opportunities.

Q56. How is the problem related to budget allocation?
Treat each project as an item, its cost as weight, its expected benefit as profit, and the available budget as capacity.

Q57. How is it used in cargo loading?
Each package has a weight and value; choose packages to maximize total value without exceeding the vehicle's weight limit.

Q58. What is the difference between a feasible solution and an optimal solution?
A feasible solution satisfies the capacity constraint. An optimal solution is a feasible solution with the greatest possible total profit.

Q59. Why does the algorithm consider every capacity from 0 to W?
The optimal choice for the full capacity may depend on a smaller remaining capacity after including an item, so the table must contain answers for all capacities.

Q60. What does the vector> dp declaration create?
It creates a dynamic two-dimensional vector with n+1 rows and capacity+1 columns, initialized to zero.

Last-minute revision table

Topic	Remember
Problem type	0/1: select whole item or leave it.
Technique	Dynamic Programming – bottom-up tabulation.
DP state	dp[i][w] = best profit with first i items and capacity w.
Choices	Exclude current item OR include it if it fits.
Decision	Take max(exclude, include).
No reuse	Include case reads from previous row i–1.
Answer	dp[n][capacity].
Reconstruction	Compare dp[i][w] with dp[i–1][w]; reduce w if selected.
Time / space	O(nW) time; O(nW) table space.

Topic	Remember
Greedy	Not guaranteed optimal for 0/1; optimal for fractional under standard assumptions.
Space optimization	1D DP, descending capacity updates; O(W) space.